ENGINEERING RECORD

PUA Square Braille

From Unicode Braille semantics to a seamless, text-capable terminal raster font

Technical handoff • Verified Linux reference implementation • 31 July 2026

Build host: tara@192.168.1.15 | Project: /home/tara/dev/FontMaker

> **Current outcome** Three preserved font families: original graphics, seam-corrected graphics, and seam-corrected graphics with ordinary text.

How to use this record

This document is designed to restore working context quickly. Read the Current State section first, then use the architecture and diagnostic chapters to understand why the implementation looks the way it does. The appendices hold exact measurements, commands, checksums, and caveats.

Contents

> **Evidence convention** “Proven” means measured or exhaustively checked in the Ubuntu/MATE reference environment. “Portable” means emitted as standard TTF/OTF and installable on the named platforms; identical raster output on macOS and Windows has not yet been captured.

1. Current state at a glance

The project now has three deliberately separate font families. No final build replaced the prior one. This preservation strategy makes regression comparison possible and prevents a text-capable experiment from silently changing the graphics-only baseline.

Family	Purpose	Installed TTF	MATE profile
PUA Square Braille	Original exact-boundary PUA graphics; blank ASCII metric anchors	~/.local/share/fonts/PUA-Square-Braille.ttf	profile4
PUA Square Braille Seamless	Graphics-only final; 60-unit outer boundary overlap	~/.local/share/fonts/PUA-Square-Braille-Seamless.ttf	profile5
PUA Square Braille Text Seamless	Seamless PUA graphics plus ordinary DejaVu-derived text	~/.local/share/fonts/PUA-Square-Braille-Text-Seamless.ttf	profile6

Recommended operational font Use PUA Square Braille Text Seamless for an interactive shell that must show both normal text and PUA graphics. Use the graphics-only seamless family for the most isolated rendering tests.

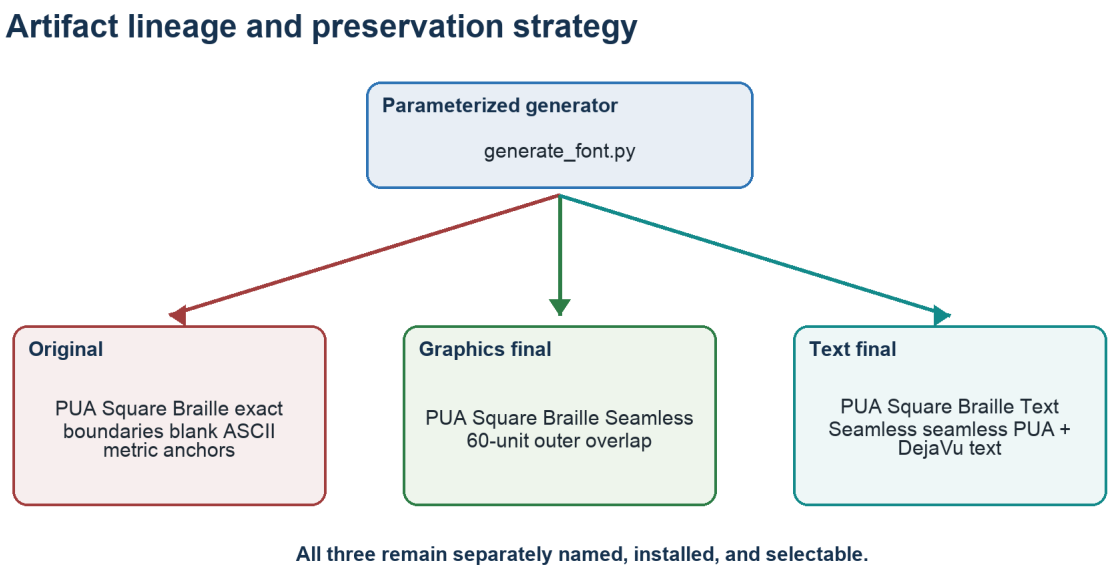

Figure 1 — Font lineage. Each stage remains separately named and recoverable.

Immediate launch commands

```
/home/tara/dev/FontMaker/launch_pua_text_terminal.sh shell
/home/tara/dev/FontMaker/launch_pua_text_terminal.sh triangle
/home/tara/dev/FontMaker/launch_pua_text_terminal.sh starfield
/home/tara/dev/FontMaker/launch_pua_text_terminal.sh trail
```

2. Original intention and acceptance criteria

The starting point was the official Unicode Braille Patterns block, named “Braille Patterns” and spanning U+2800–U+28FF. That block contains every possible state of eight Braille positions. Its bit semantics were to be preserved, but the rendered shape was intentionally changed.

> **Original visual requirement** Replace each Braille dot position with a square position. Each square must occupy its full equal subcell so adjacent occupied positions have no intentional gap horizontally or vertically, including across terminal-character boundaries.

Functional requirements

- Generate the font programmatically, with geometry and mapping parameters exposed in source.
- Emit desktop font formats usable by Linux, Windows, and macOS.
- Place the custom glyphs in a Unicode Private Use Area rather than redefining U+2800–U+28FF.
- Retain a one-to-one offset relationship with all 256 official Braille patterns.
- Turn one terminal character cell into eight addressable occupancy positions: 2 columns × 4 rows.
- Allow a virtual occupancy canvas of (2 × terminal columns) by (4 × terminal rows).
- Provide objective visual and programmatic evidence rather than relying on a FontForge glyph-browser view.

Acceptance criteria that emerged during testing

- A full U+E0FF field must form a visually continuous mass without cell-border gridlines.
- The mapping must remain correct for every value from 0 through 255.
- Interactive and animated demos must exhibit sub-character movement without fallback-glyph corruption.
- The final interactive font must also display ordinary text, numbers, punctuation, and broad common-script coverage.
- The original font must remain intact while corrected variants are developed.

3. Conceptual architecture and mapping

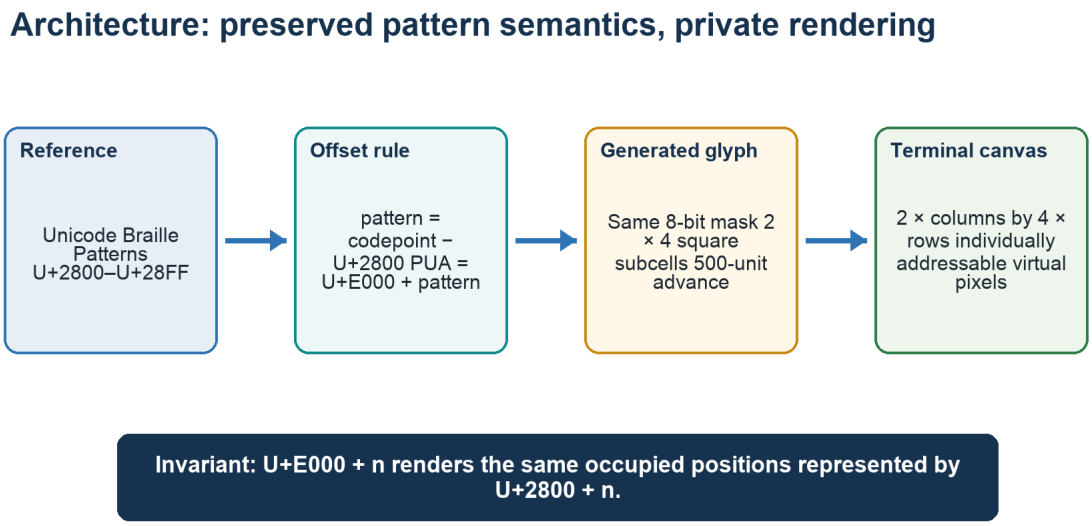

Figure 2 — Unicode pattern semantics are preserved while rendering is moved to a private codepoint range.

3.1 Offset rule

For any Braille pattern value n in the inclusive range 0…255:

```
official_codepoint = 0x2800 + n
private_codepoint  = 0xE000 + n
character          = chr(private_codepoint)
```

The custom font therefore occupies U+E000–U+E0FF. It does not redefine the standard Unicode Braille block. A text stream containing official Braille still requests U+2800–U+28FF; a graphics stream must explicitly emit the corresponding PUA characters.

3.2 Braille dot numbering and cell geometry

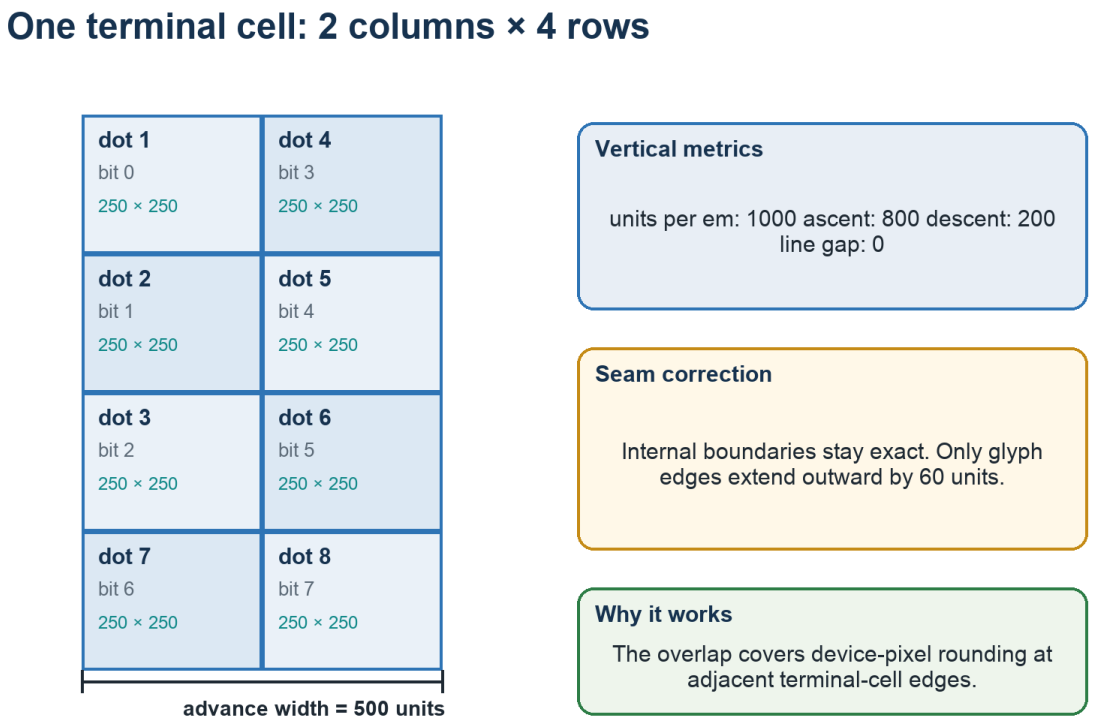

Figure 3 — Normative 2×4 cell geometry, dot numbering, metrics, and edge-only overlap.

The dot-number matrix is ((1,4), (2,5), (3,6), (7,8)). Pattern bit k controls dot k+1. The 500-unit advance divides into two 250-unit columns. The 1000-unit em divides into four 250-unit rows. Ascent 800 places the top at y=800; descent 200 places the bottom at y=−200.

3.3 Packing terminal pixels into glyph masks

```
cell_x = pixel_x // 2
cell_y = pixel_y // 4
local_column = pixel_x % 2
local_row = pixel_y % 4
dot_number = ((1,4), (2,5), (3,6), (7,8))[local_row][local_column]
mask |= 1 << (dot_number - 1)
output_character = chr(0xE000 + mask)
```

Color granularity Occupancy has 2×4 subcell resolution, but a conventional terminal applies one foreground color to the whole character cell. The demonstrations can move edges at sub-character resolution; independent colors for two occupied subcells in the same cell are not available in one rendering pass.

4. Build and test environment

The authoritative build and runtime tests were performed on the user's Linux system through the already-authorized SSH path. macOS supplied screenshots and retained deliverables, but the pixel-level terminal measurements were made on the Linux reference renderer.

Layer	Reference environment / version	Role
Host	tara@192.168.1.15	Build and graphical test account
Project	/home/tara/dev/FontMaker	Sources, binaries, probes, captures, archives
Operating system	Ubuntu 24.04.4 LTS; Linux 6.8.0-136-generic aarch64	Reference platform
FontForge	20230101 package 1:20230101~dfsg-1.1build1	Font construction; TTF, OTF, SFD output
Python	3.12.3	Generators, demos, analyzers, PTY regression harnesses
fontTools	4.46.0	Compiled table, cmap, metric, and outline inspection
Fontconfig	2.15.0	User-font discovery, selection, and cache verification
MATE Terminal	1.26.1	Dedicated font profiles and actual terminal rendering
VTE / Pango	VTE 0.76.0; Pango 1.52.1	Terminal layout and glyph rendering path
ImageMagick	6.9.12-98	Window-scoped capture through import
Pillow	10.2.0 on server	Pixel classification and connected-component measurement
Shell tools	ssh, scp, dconf, gsettings, script, timeout	Remote orchestration, profiles, PTY interaction

Truth-finding strategy

1. Construct deterministic probes that render one known glyph or one known terminal background.
2. Capture only the test terminal window using its WINDOWID.
3. Classify pixels numerically, not by screenshot impression alone.
4. Change one variable at a time: size, antialiasing, hinting, outline format, then edge overfill.
5. Retain all candidate files and captures while removing temporary candidates from the active font registry.
6. Run exhaustive glyph and regression tests before installing a separately named final family.

5. From conception to first working font

5.1 Generator design

generate_font.py creates a UnicodeFull FontForge font, defines metrics, adds a .notdef glyph, adds printable ASCII metric anchors, and emits every PUA pattern. For each pattern it identifies occupied 2×4 cells, groups 4-connected cells, and traces only the external boundary of each connected component. This avoids internal seams inside one glyph.

The core generation parameters are pua_start, width, ascent, descent, version, edge_overfill, font_name, output_dir, and an experimental autoinstruct flag. Default generation remains compatible with the original exact-boundary font because edge_overfill defaults to zero.

```
fontforge -lang=py -script generate_font.py \
  --font-name "PUA Square Braille Seamless" \
  --version 1.1 --edge-overfill 60 --output-dir dist
```

5.2 Initial artifacts

Format	Purpose	Initial file
TTF	Recommended desktop/terminal distribution	dist/PUA-Square-Braille.ttf
OTF	Alternative standard outline distribution	dist/PUA-Square-Braille.otf
SFD	Editable FontForge source	dist/PUA-Square-Braille.sfd
HTML	Browser specimen with visible codepoint labels	specimen.html

5.3 Why FontForge's glyph window was insufficient

Opening the TTF in FontForge showed glyph cells but did not provide a clear operational proof of which PUA codepoint was being rendered in the terminal. A specimen page and deterministic terminal probes were therefore created. The principle was simple: the evidence screen must label offsets or use a known generated sequence, and the terminal itself must actually select the target family.

> **Early failure mode** If the terminal profile did not select the custom family, U+E000–U+E0FF fell through to unrelated fonts. The result was rows of symbols, icons, replacement shapes, or blanks rather than square cells. Installing a font with fc-cache was necessary but not sufficient; the terminal profile also had to request the family.

6. Terminal demonstrations and visual validation

The demos evolved from diagnosis into acceptance tests. Each one exercised a different property of the virtual framebuffer and exposed errors that a static specimen could hide.

Program	Purpose	Evidence provided
geometry_test.py	Alternating colors, diagonals, fills	Font selection, 2×4 mask mapping, obvious spacing
snow.py	Falling particles	Animated sub-character vertical motion; fallback corruption detection
starfield.py	3D fly-through	Smooth movement, repeated diagonal traces, full-screen stability
trail.py	Arrow-key point with persistent trail	Interactive pixel addressing and boundary crossings
triangle.py	Filled 180×120 virtual-pixel RGB triangle	Large solid mass; most sensitive gridline detector
text_probe.py	Normal text, Unicode fallback, PUA samples, full strip	Text-capable final-family proof

The trail demonstration initially appeared to have a new vertical spacing defect. The defect was traced to its coordinate-to-cell calculation rather than to the starfield-tested font. After correction, drawn outlines crossed horizontal and vertical terminal-cell boundaries correctly.

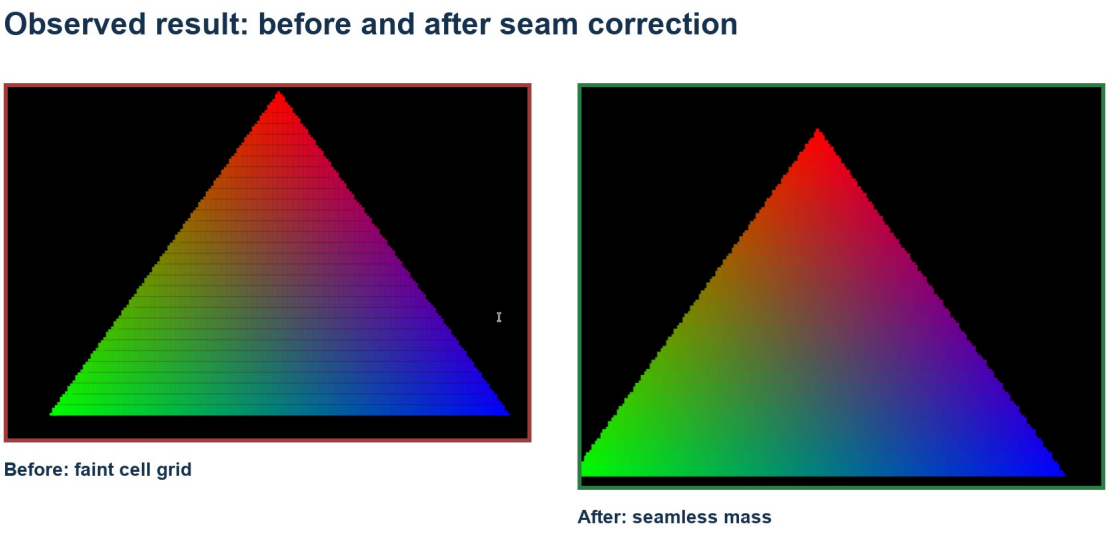

Figure 4 — User-observed triangle before and after the seam correction.

7. The faint-gridline investigation

The critical defect was a faint rectangular grid visible at character-cell borders when a large solid mass was drawn. Because the glyph geometry nominally filled its 500×1000 cell, the cause was not obvious from source coordinates alone.

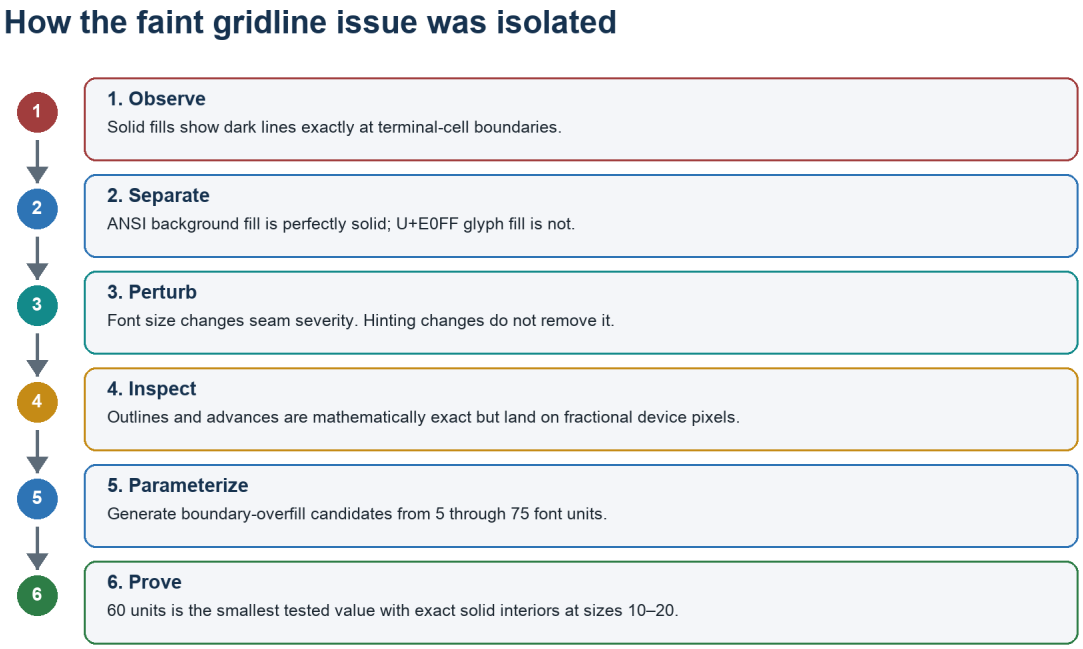

Figure 5 — Evidence chain used to isolate and fix the cell-border gridlines.

7.1 The decisive separation test

seam_probe.py rendered two otherwise equivalent full fields. Glyph mode painted repeated U+E0FF in magenta. Background mode painted ordinary spaces with an ANSI magenta cell background. If terminal cell layout itself contained spacing, both should have shown the grid.

Controlled full-field evidence

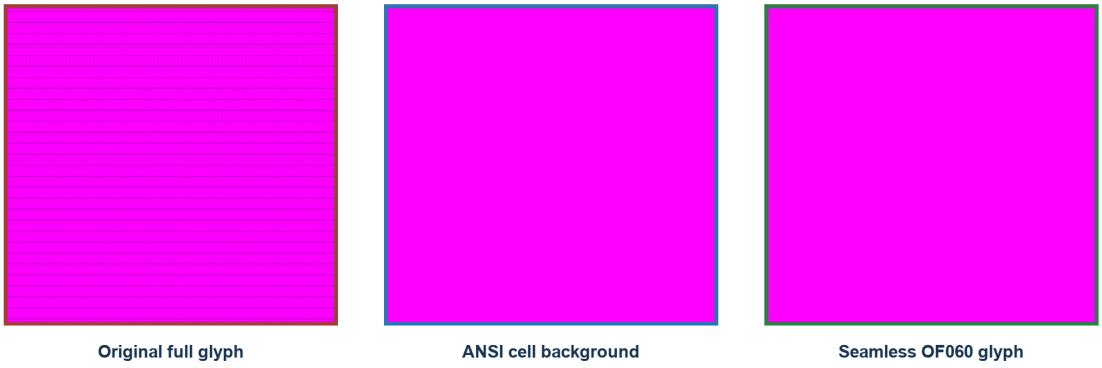

Figure 6 — Original glyph fill versus terminal background fill versus the final OF060 glyph.

Probe	Exact magenta	Interpretation
Original U+E0FF glyph field	69.712947%	Many edge pixels were darkened by rasterization
ANSI background field	100%	Terminal cell allocation and background painting were seamless
Final OF060 interior, sizes 10–20	100%	Boundary overlap covered raster rounding at tested sizes

Root-cause conclusion The terminal did not insert physical gaps. Exact outline edges landed on fractional device pixels, and antialiasing blended those outer glyph pixels with the black background. Repeating the glyph made the blended edges appear as a grid.

7.2 Size, antialiasing, and hinting tests

Changing the terminal font size changed seam intensity, which is characteristic of outline-to-device-pixel rounding. Disabling antialiasing did not solve the problem; it converted blended seams into black gaps. Grayscale, full-hint, and unhinted configurations did not materially improve the baseline. Automatic FontForge instructions did not produce useful per-glyph TrueType programs for this geometry.

Size	Exact magenta	Mean loss
10	61.31%	25.59
12	66.31%	20.99
14	69.59%	6.01
16	74.99%	26.32
18	76.77%	14.15
20	78.67%	12.82

7.3 Compiled-font inspection

fontTools and FontForge inspection confirmed the intended values: unitsPerEm 1000; advance 500; ascent 800; descent 200; line gap 0; U+E0FF bounding box 0,−200 to 500,800 in the original; fixed-pitch metadata; and no effective glyph instructions. The outline was mathematically exact. The missing piece was renderer behavior at fractional pixel boundaries.

7.4 Parameterized boundary-overfill sweep

The generator was extended with edge_overfill. It alters only contours on the four outer glyph boundaries. It does not alter the internal 250-unit subcell partitions. Candidates of 5, 10, 15, 20, 25, 30, 40, 50, 60, 65, and 75 units were generated under unique names and tested without replacing the original.

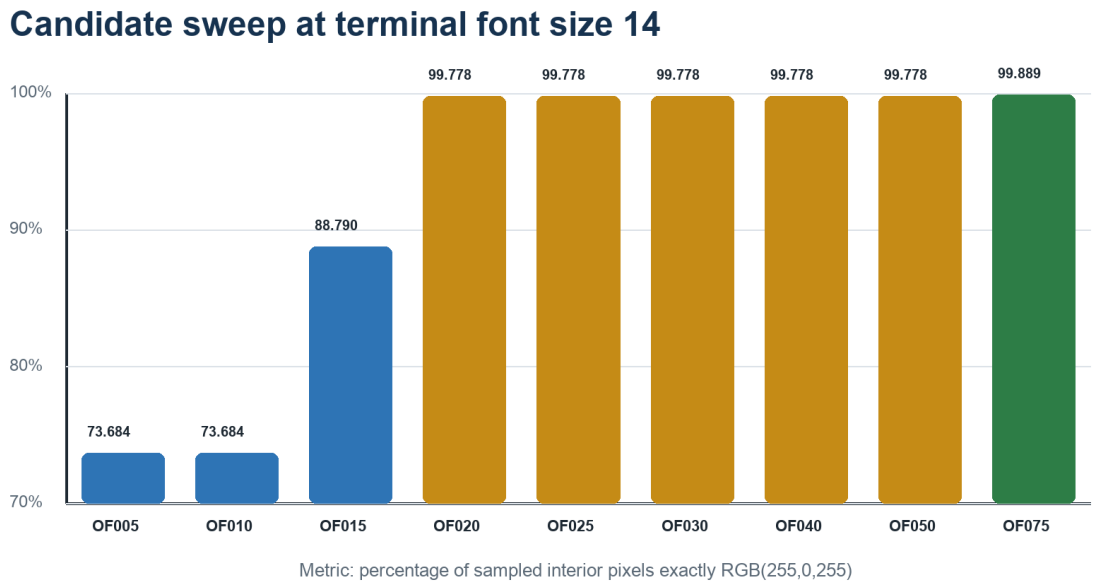

Figure 7 — Candidate sweep at size 14; OF020 nearly closes the field, while cross-size testing selected OF060.

OF020 appeared seamless at the target size 14, but it failed exact-interior tests at several other sizes. OF050 was exact at sizes 12–20 but not size 10. OF060 was the smallest tested candidate that produced a 100% exact-magenta interior at all six sizes 10, 12, 14, 16, 18, and 20. OF065 and OF075 also passed but introduced more overlap than necessary.

7.5 Trade-off measurement

Overfill necessarily expands an isolated edge-touching square. Connected-component measurement on a repeated sparse U+E008 field gave a baseline device-pixel box of 4×5 with area 20, OF020 of 5×5 with

area 25, and OF060 typically 6×6 with median classified area 35. This is the explicit trade-off: guaranteed continuity for solid adjacency at the cost of controlled bleed into a neighboring empty cell.

> **Final graphics solution** Use a 60-font-unit outward extension on outer glyph boundaries only. Keep internal 2×4 partitions exact. Preserve the original zero-overfill font as the reference build.

8. Final font lineage and text-capable extension

8.1 Why the original shell was blank

The graphics font originally encoded blank printable-ASCII glyphs as metric anchors. VTE derives terminal cell geometry from ordinary printable characters; the anchors prevented a fallback font with different advances or line metrics from deciding the graphics grid. The consequence was intentional: an interactive shell using that family could not display normal alphanumeric text.

8.2 Combined text design

add_text_to_font.py opens the seam-corrected TTF, opens the system DejaVu Sans Mono source, normalizes the text source to the 1000-unit em, and horizontally scales text outlines into the established 500-unit terminal advance. It excludes U+E000–U+E0FF so the verified graphics outlines remain untouched. The result is separately named PUA Square Braille Text Seamless, style Regular.

```
fontforge -lang=py -script add_text_to_font.py \
  --graphics-font dist/PUA-Square-Braille-Seamless.ttf \
  --text-font /usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf \
  --font-name "PUA Square Braille Text Seamless" \
  --version 1.2 --output-dir dist
```

The build copied 3,322 DejaVu source glyphs. The compiled TTF contains 3,581 glyphs and 3,578 cmap entries: 3,322 text mappings plus 256 PUA graphics mappings. Printable ASCII characters are non-empty and have a 500-unit advance. The DejaVu license is preserved in LICENSE-DejaVu.txt.

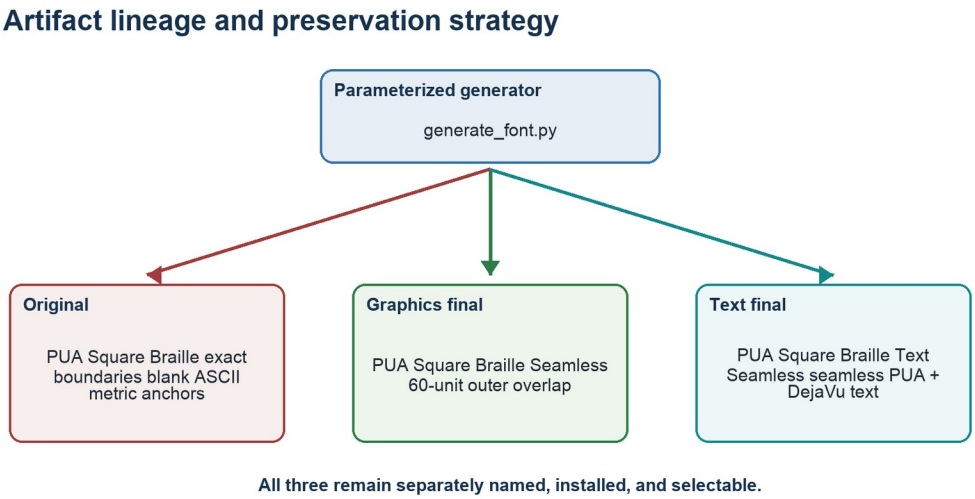

Figure 8 — Preservation is structural: the text-capable font extends the seamless build instead of replacing it.

8.3 Current metadata note

> **Metadata discrepancy** The text compositor was invoked with version 1.2, but the compiled name table currently reports “Version 1.1”, inherited from the graphics source. Functionality and filenames are unaffected. A future metadata-only rebuild should explicitly replace name ID 5 if version labeling matters.

9. What has been proven

9.1 Exhaustive structural proof

- Every U+E000–U+E0FF codepoint exists in TTF and OTF output.
- Every PUA glyph has advance width 500 and the expected bounding box for its occupied positions and 60-unit edge overlap.
- All 256 PUA outlines in the text-capable font match the graphics-only seamless font through recording-pen comparison.
- Printable ASCII U+0020–U+007E exists; every non-space printable ASCII glyph has a non-empty outline and 500-unit advance.
- Representative Latin, Greek, Cyrillic, currency, arrow, box-drawing, and block characters are present.
- Fontconfig selects each installed family by its exact separate name.

9.2 Pixel proof

For the final seamless graphics family and the text-capable family, the interior of controlled U+E0FF terminal captures was exactly RGB(255,0,255) at sizes 10, 12, 14, 16, 18, and 20. Maximum channel error in the analyzed interior was zero.

9.3 Behavioral proof

triangle.py, starfield.py, and trail.py completed under controlled pseudo-terminal regression. trail.py received delayed automated arrow-key input after entering raw mode, avoiding an initial harness timing false failure. The user's final triangle screenshot independently confirmed that the visible grid had disappeared.

9.4 Preservation proof

The original TTF checksum remained unchanged throughout the seam and text work; the complete SHA-256 value is recorded in Appendix E. The original, graphics-final, and text-final families are simultaneously installed under profile4, profile5, and profile6 respectively.

> **Scope of proof** Pixel-perfect continuity is proven on Ubuntu 24.04 MATE Terminal/VTE/Pango/Cairo at the six tested sizes. TTF and OTF portability is structural; an equivalent screenshot matrix has not yet been run on macOS Terminal, iTerm2, Windows Terminal, DirectWrite, or CoreText.

10. Operations, installation, and reproduction

10.1 Linux launchers

Launcher	Family	Modes
launch_pua_terminal.sh	PUA Square Braille	shell, snow, starfield, trail, triangle, test, setup
launch_pua_seamless_terminal.sh	PUA Square Braille Seamless	shell, snow, starfield, trail, triangle, test, setup
launch_pua_text_terminal.sh	PUA Square Braille Text Seamless	shell, snow, starfield, trail, triangle, test, text-test, setup

```
cd /home/tara/dev/FontMaker
./launch_pua_text_terminal.sh setup
./launch_pua_text_terminal.sh shell
./launch_pua_text_terminal.sh triangle
```

10.2 Validate Linux installation and selection

```
fc-match -f 'Selected file: %{file}\nFamily: %{family}\nStyle: %{style}\n' \
  'PUA Square Braille Text Seamless'

fc-list -f '%{file}\t%{family}\n' | grep -F 'PUA Square Braille'
```

10.3 macOS

Install the TTF through Font Book, or copy it to ~/Library/Fonts/. Then select PUA Square Braille Text Seamless in the chosen terminal profile. Use the TTF or OTF, not both. The font is structurally portable, but repeat the seam probe at relevant point sizes before assuming the Linux OF060 threshold is optimal under CoreText.

10.4 Windows

Right-click the TTF and choose Install, or Install for all users where appropriate. In Windows Terminal select PUA Square Braille Text Seamless under profile Appearance → Font face. Again, install only one outline format and run a Windows-native solid-fill test because DirectWrite rasterization can differ from Pango/Cairo.

10.5 Rebuild sequence

1. Run generate_font.py with the desired PUA start, cell metrics, and edge overfill.
2. Run verify_overfill_matrix.py against generated graphics TTF/OTF candidates.
3. Run add_text_to_font.py if ordinary text is required.
4. Run verify_text_font.py to compare every PUA outline and verify printable ASCII.
5. Copy only the chosen TTF to ~/.local/share/fonts and refresh with fc-cache -f.
6. Configure a dedicated terminal profile and run the six-size seam matrix and demos.

11. File locations and artifact inventory

11.1 Authoritative server project

Base directory: /home/tara/dev/FontMaker

Path	Contents / significance
dist/PUA-Square-Braille.{ttf,otf,sfd}	Preserved original exact-boundary graphics font
dist/PUA-Square-Braille-Seamless.{ttf,otf,sfd}	Graphics-only 60-unit-overfill final
dist/PUA-Square-Braille-Text-Seamless.{ttf,otf,sfd}	Text-capable seamless final
dist/candidates/	All parameter candidates and hinted/CFF experiments
test-artifacts/registered-candidates/	Recoverable copies removed from the active font registry
generate_font.py	Parameterized 2×4 PUA graphics generator
add_text_to_font.py	DejaVu-derived text compositor

Path	Contents / significance
verify_font.py; verify_overfill_matrix.py; verify_text_font.py	Structural and outline verification
seam_probe.py; sparse_probe.py; text_probe.py	Deterministic terminal evidence probes
run_*matrix.sh; run_*regression.sh	Automated graphical test harnesses
snow.py; starfield.py; trail.py; triangle.py	Demonstrations and behavioral regression programs
README.md; INSTALL.md; TEXT-FONT.md; LICENSE-DejaVu.txt	Usage, installation, final handoff, and licensing

11.2 Active user fonts

```
/home/tara/.local/share/fonts/PUA-Square-Braille.ttf
/home/tara/.local/share/fonts/PUA-Square-Braille-Seamless.ttf
/home/tara/.local/share/fonts/PUA-Square-Braille-Text-Seamless.ttf
```

11.3 Local deliverables

Conversation deliverables are retained under:

```
/Users/tara/Documents/Codex/2026-07-29/i-want-you-to-create-me/outputs/PUA-Square-
Braille/
/Users/tara/Documents/Codex/2026-07-29/i-want-you-to-create-me/outputs/PUA-Square-
Braille-Seamless/
/Users/tara/Documents/Codex/2026-07-29/i-want-you-to-create-me/outputs/PUA-Square-
Braille-Text-Seamless/
```

11.4 VROBI configuration

The original configuration is /Users/tara/Documents/VROBI-braille-new.cfg. The PUA-translated counterpart is /Users/tara/Documents/VROBI-braille-new-PUA.cfg, with a retained copy in outputs/VROBI-braille-new-PUA.cfg. All 256 array indices are preserved; only the requested character is translated from U+2800+n to U+E000+n.

12. Limitations and open engineering notes

- PUA meaning is private by definition. A consumer must share the U+E000–U+E0FF convention and select this font.
- The 60-unit overlap can enlarge isolated edge-touching pixels and may intrude visually into an intentionally empty neighbor. It was selected as the smallest tested value robust across sizes 10–20 in the reference renderer.
- The combined font copies glyph outlines, not the full DejaVu shaping system. FontForge reported lost anchor classes during composition. The verified target is terminal-oriented normal text, not full complex-script typography.

- Official Unicode Braille U+2800–U+28FF is a semantic reference and is not remapped inside this font. In the mixed proof it is supplied by fallback because U+2801 is absent from the final cmap.
- The combined font head bounding box spans approximately x=−464…597 and y=−375…1028 because of broad source coverage, while terminal metrics remain ascent 800/descent 200. Rare imported glyphs outside those metrics may clip; printable ASCII and demonstrated common text were verified.
- FontForge validation reports self-intersection and missing-extrema flags inherited from intentional overlap geometry and source outlines. fontTools parsing, exhaustive PUA comparison, installation, and rendering tests pass.
- The compiled text font name table currently says Version 1.1 even though the compositor command accepted 1.2. Correct name ID 5 in a future metadata cleanup.
- Color is character-cell scoped in conventional terminals even though occupancy is sub-character scoped.
- Cross-platform installation is documented, but pixel-identical rendering is not yet proven outside the Linux MATE/VTE/Pango/Cairo reference stack.

> **Recommended next verification** Run seam_probe.py or an equivalent full U+E0FF field on macOS and Windows at the terminal sizes actually used. If a renderer shows bleed or gaps, select a platform-specific overfill value rather than changing the preserved OF060 Linux build.

Appendices

Appendix A. Normative font specification

Property	Final text-capable value
Family / style	PUA Square Braille Text Seamless / Regular
PostScript name	PUASquareBrailleTextSeamless-Regular
Compiled version string	Version 1.1 (see metadata note)
Formats	TTF 350,216 bytes; OTF 846,496 bytes; SFD 2,971,473 bytes
Units per em	1000
Advance width	500
Ascent / descent / line gap	800 / 200 / 0
Subcell geometry	250 × 250 units; 2 columns × 4 rows
PUA range	U+E000–U+E0FF inclusive, 256 patterns
Reference range	Unicode Braille Patterns U+2800–U+28FF
Edge overlap	60 units on occupied outer boundaries only
Internal partitions	Exact at x=250 and y=550,300,50 in font coordinates
Glyph count / cmap count	3,581 glyphs / 3,578 mapped codepoints
Monospace metadata	post.isFixedPitch = 1; OS/2 weight 400
hhea metrics	ascent 800; descent −200; lineGap 0; advanceWidthMax 500
OS/2 typo metrics	800 / −200 / 0; USE_TYPO_METRICS set
Global head bbox	x −464…597; y −375…1028
gasp	Range through 65535; behavior value 2
License	Generated PUA outlines; text outlines under included Bitstream Vera/DejaVu notice

Appendix B. Braille and VROBI bit mapping

The VROBI array index uses row-major virtual-pixel bit order, while Unicode Braille assigns bits by dot number. The configuration therefore does not simply use U+2800 + index. It permutes index bits into Unicode pattern bits, then the PUA file changes only the base from 0x2800 to 0xE000.

VROBI index bit	Grid position	Braille dot	Pattern bit
0	row 0, column 0	1	0
1	row 1, column 0	2	1
2	row 2, column 0	3	2
3	row 3, column 0	7	6
4	row 0, column 1	4	3
5	row 1, column 1	5	4
6	row 2, column 1	6	5
7	row 3, column 1	8	7

```
pattern = (index & 0x07) | ((index & 0x08) << 3) \
        | ((index & 0x70) >> 1) | (index & 0x80)
original_character = chr(0x2800 + pattern)
pua_character      = chr(0xE000 + pattern)
```

A programmatic audit confirmed this relationship for all 256 lines in VROBI-braille-new.cfg and VROBI-braille-new-PUA.cfg.

Appendix C. Measurement tables

C.1 Size sensitivity of the original exact-boundary glyph

Font size	Exact magenta	Mean channel loss
10	61.31%	25.59
12	66.31%	20.99
14	69.59%	6.01
16	74.99%	26.32
18	76.77%	14.15
20	78.67%	12.82

C.2 Overfill candidate sweep at size 14

Candidate	Exact magenta	Mean loss
OF005	73.684%	3.228
OF010	73.684%	0.849

Candidate	Exact magenta	Mean loss
OF015	88.790%	0.168
OF020	99.778%	0.085
OF025	99.778%	0.063
OF030	99.778%	0.060
OF040	99.778%	0.119
OF050	99.778%	0.067
OF075	99.889%	0.017

C.3 Cross-size finalist exactness

Candidate	10	12	14	16	18	20
OF020	61.46%	70.74%	100%	74.95%	88.13%	93.05%
OF030	71.47%	82.50%	100%	83.25%	96.10%	100%
OF050	86.75%	100%	100%	100%	100%	100%
OF060	100%	100%	100%	100%	100%	100%
OF065	100%	100%	100%	100%	100%	100%
OF075	100%	100%	100%	100%	100%	100%

C.4 Sparse-pixel enlargement

Build	Median device bbox	Median classified area
Original	4 × 5	20
OF020	5 × 5	25
OF060	typically 6 × 6	35

Appendix D. Command reference

D.1 Inspect and validate installed fonts

```
fc-match 'PUA Square Braille Text Seamless'
fc-scan ~/.local/share/fonts/PUA-Square-Braille-Text-Seamless.ttf
fc-list -f '%{file}\t%{family}\n' | grep -F 'PUA Square Braille'
```

D.2 Build the graphics fonts

```
cd /home/tara/dev/FontMaker
fontforge -lang=py -script generate_font.py --output-dir dist
fontforge -lang=py -script generate_font.py \
  --font-name 'PUA Square Braille Seamless' --version 1.1 \
  --edge-overfill 60 --output-dir dist
```

D.3 Build and verify the text-capable font

```
fontforge -lang=py -script add_text_to_font.py \
  --graphics-font dist/PUA-Square-Braille-Seamless.ttf \
  --text-font /usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf \
  --font-name 'PUA Square Braille Text Seamless' --version 1.2 \
  --output-dir dist

python3 verify_text_font.py \
  dist/PUA-Square-Braille-Seamless.ttf \
  dist/PUA-Square-Braille-Text-Seamless.ttf \
  dist/PUA-Square-Braille-Seamless.otf \
  dist/PUA-Square-Braille-Text-Seamless.otf
```

D.4 Run regression suites

```
DISPLAY=:0 DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus \
  ./run_seamless_regression.sh
DISPLAY=:0 DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus \
  ./run_text_regression.sh
```

Appendix E. Checksums and provenance

Artifact	SHA-256
PUA-Square-Braille.ttf	d0194dbbefdc2fb102b2829eaa2d20e567cf61f250909447ed3c8263717ab3c3
PUA-Square-Braille.otf	1c32e6def6b166bf07048db5c12e70d8cd5e0a0b9bed51a7e9a77ba3da059b4a
PUA-Square-Braille.sfd	1ff448f5df0aee87fa03cbfca30f61a8c8ffa48107c73cd6c5f4ca3a6e0e30ce
PUA-Square-Braille-Seamless.ttf	587e3466a026d9f3fd3d11fed1248a829bdb65dfef841f20112ebf6531c6aed0
PUA-Square-Braille-Seamless.otf	d5ef45c72863562f100adec313b9b0ae1a16be6e63ab53b4149ae9b7bd5d877d
PUA-Square-Braille-Text-Seamless.ttf	4f2ad7654915e81811b43eee432946e3bb2821ddf86ab105614b59d1127cf9ad
PUA-Square-Braille-Text-Seamless.otf	548bfaa1597d4dd4bd83b54fe5d1a74c272b72c3de7124035cc3ee2b388dba9a

The original TTF checksum was recorded before candidate work and rechecked after final installation. Candidate fonts remain under dist/candidates and test-artifacts; they are not active user fonts.

Appendix F. Toolchain and sources

S1. Unicode Names List — Braille Patterns U+2800–U+28FF: Open source S1

S2. Unicode code chart PDF — U2800: Open source S2

S3. FontForge Python scripting documentation: Open source S3

S4. FontForge font/selection API documentation: Open source S4

S5. DejaVu/Bitstream Vera license: `/home/tara/dev/FontMaker/LICENSE-DejaVu.txt`

S6. Project evidence: `/home/tara/dev/FontMaker/*.png, run_*matrix.sh, run_*regression.sh`

> **Handoff completeness** This record, the final binaries, editable SFD files, generators, verifiers, launchers, license, VROBI mapping, and evidence captures together contain enough context to reproduce the implementation and resume platform-specific validation without reconstructing the investigation.